

Tatami — Phase 1

JSON to OCaml Modules: Lowering Scenarios

Durwasa Chakraborty

Vimala Soundarapandian

Vishakh Desai

Design document — working draft

1 Purpose and scope

This document fixes the meaning of Phase 1: the translation from JSON input to a set of generated OCaml modules. It does not cover the Phase 2 effect lowering.

The design note already fixes the relational-to-OCaml half of the translation: table becomes module, row becomes a value of type `t`, column becomes a record field, primary key becomes an abstract `id`, foreign key becomes a field holding another module's `id`, joining on it becomes an accessor function, and a nullable column becomes an `option` field. Nothing in that correspondence is open.

What this document settles is the other half — JSON to relational — and the shape of the module surface that results. Each scenario below states the options that were considered, with an example of each, and then records which one was selected.

Delivery. The tool is intended to be used through a web interface: JSON is supplied directly, and the generated modules are presented in an editor-like view. Two consequences bear on the decisions below. Any configuration a decision depends on — the path marking of S9, for instance — must be a persisted, exportable input rather than interface state, so that the same input regenerates the same modules outside the interface. And the generated modules are read by people in an editor rather than consumed as a build artifact, which raises the weight of the naming decisions deferred to S22. An interface that invites a single pasted document will also routinely produce the weaker schema described in S0.

The unit of inference is the path, across all documents. Throughout, a scenario describes *the set of values observed at a path across the whole input*, not the value at a path in one document. A path is a position in the schema, written `.price` or `.items[].qty`. Several scenarios are invisible at single-document granularity: a field that is an integer in one document and a string in another, or an array that happens to be empty everywhere.

2 Decision status

Ref	Scenario	Status
S0	Unit of input	decided
S1a	Numbers: integer versus fractional	decided
S1b	Integers outside native <code>int</code> range	decided
S2	Strings	decided
S3	Booleans	decided
S4	Explicit <code>null</code> and absent fields	decided
S6	Field never given a type	decided
S7	Nested object, one-to-one	decided
S8	Empty object	decided
S9	Map-shaped object (keys are data)	decided
S10	Duplicate keys	decided
S11	Key names that are not OCaml identifiers	decided
S13	Array of objects	decided
S14	Array of scalars	decided
S15	Empty arrays	decided
S16	Heterogeneous array	decided
S16b	Null elements in an array	decided
S17	Array of arrays	decided
S18	Field with differing types across documents	decided
S19	Recursive and mutually recursive shapes	decided
S20	Required versus optional inference	decided
S21.1	Abstractness of <code>id</code>	decided
S21.2	The store behind <code>get</code>	decided
S21.3	Presence marker in the record type	decided
S21.4	Foreign key field naming	decided
S21.5	Entry point into the modules	decided
S21.6	Child back-reference in the record	decided
S22	Name mangling and collisions	decided

(S12, “root value that is not an object”, has been absorbed into S0; S5, “field absent in some documents”, into S4.)

3 Input

S0. Unit of input

JSON reaches the tool in more than one shape, and the shapes disagree about what counts as one document. The distinction matters because inference is over the set of values at a path: a root field is observed once per document, so how many documents there are determines how much can be inferred.

Two groups of input form must be separated. The first group changes what counts as a document and therefore needs a decision:

- **Single document** — one JSON value.
- **Top-level array** — $[\{\dots\}, \{\dots\}]$. Ambiguous: either a corpus of several documents, or one document whose root happens to be an array.
- **JSON Lines / NDJSON** — one JSON value per line. Unambiguously a corpus.
- **Concatenated JSON** — $\{\dots\}\{\dots\}$ with no delimiter. A corpus, but requires a streaming parser.

- **Wrapped corpus** — `{"meta":{"..."},"records":[...]}`, where the documents live at a nested path rather than at the root.

The second group is plumbing: a directory of files, several files on the command line, standard input, compression, character encoding. These change how bytes reach the parser and nothing about inference, so they carry no design decision and can be added at any point.

Options considered.

- (a) Single document only. Inference from one JSON value.
- (b) Corpus only. The input is always many documents.
- (c) Both, with a top-level array always read as a corpus. ← **selected**
- (d) Both, disambiguated by an explicit `--corpus` / `--single` flag.

Decision. Accept a single document and a top-level array. A top-level array is always read as a *corpus of its elements*, never as one document whose root is an array. JSON Lines, concatenated JSON and wrapped corpora are deferred; they extend the input reader without disturbing any other decision in this document.

Given a single document, the corpus is that one document:

```
{ "a": 1, "b": { "c": 2 } }

-> corpus of 1 document
```

Given a top-level array, the corpus is its elements:

```
[ { "a": 1 }, { "a": 2 } ]

-> corpus of 2 documents
-> NOT one document whose root is an array
```

Rationale. Reading a top-level array as a corpus removes the ambiguity without a flag, and matches how JSON corpora are actually distributed. It also implies that the root of a document is always an object, which is why S12 no longer needs to exist as a separate scenario.

Known limitation. A document whose root is genuinely an array cannot be expressed; it must be wrapped, as `{"items": [...]}`. Separately, a single document cannot support optionality inference: every field present in it is total and no field is ever partial, so single-document input yields a strictly weaker schema than corpus input. Arrays *inside* a single document still supply many observations, so widening and optionality behave normally within them.

4 Atoms

S1a. Numbers: integer versus fractional

JSON has one number type; OCaml distinguishes `int` from `float`. A path holding numbers must be assigned one of them, and the same path may hold both integral and fractional values across the corpus.

```
{ "qty": 1 } { "qty": 2 } -- integral everywhere
{ "price": 1 } { "price": 2.5 } -- mixed at one path
```

Options considered.

- (a) **Split by value, widen on mix.** A path is `int` if every value observed there is integral, `float` otherwise. Under this reading `1.0` is integral and yields `int`.
- (b) **Split by token, widen on mix.** Follow the lexical form the parser already reports: `1` is an integer, `1.0` is a float regardless of its value. A path carrying any float becomes `float`.
← **selected**
- (c) **Every number is float.** Map all JSON numbers to `float`, matching the IEEE-double reading most JSON implementations take. No widening logic, but counts, indices and identifiers all become floats.
- (d) **Split, and reject on mix.** Keep `int` and `float` separate, and treat a path carrying both as a hard error rather than widening it.

Decision. Split by token, widen on mix. A path whose values are all written without a decimal point becomes `int`; a path where any value anywhere in the corpus is written with a decimal point becomes `float` for the whole column.

```
{ "qty": 1 }
{ "qty": 2 }
-> qty : int

{ "price": 1 }
{ "price": 2.5 }
-> price : float (widened: one float anywhere widens the column)

{ "x": 1.0 }
-> x : float (token is a float, even though the value is round)
```

Widening applies at every path, at any depth. Root fields and array-element fields obey the same rule; they differ only in how many observations they receive, since a root field is observed once per document while an array-element field is observed once per element:

```
{ "price": 1, "items": [ { "qty": 1 } ] }
{ "price": 2.5, "items": [ { "qty": 2 }, { "qty": 3 } ] }
{ "price": 3, "items": [] }

.price sees 1, 2.5, 3 -> float
.items[].qty sees 1, 2, 3 -> int
```

Rationale. The parser already draws the lexical line — `yojson` reports `‘Int` and `‘Float` as distinct constructors — so option (b) costs nothing to implement and respects the distinction the document author wrote when they wrote it. Widening rather than rejecting keeps inference monotone: adding documents to a corpus can only loosen a type, never invalidate one already inferred, which matters because the corpus is not usually known in full ahead of time.

Known limitation. The lexical signal is not always preserved by the producer. JavaScript has only doubles, and `JSON.stringify(1.0)` emits `1`, so a corpus of genuinely fractional values that all happen to be round will infer `int`. Widening cannot recover this, because the information is absent from the data. This is a limit of inferring types from values rather than a consequence of the lexical choice: in any corpus where some value is fractional, widening gives the right answer.

S1b. Integers outside native `int` range

OCaml’s native `int` is 63 bits on a 64-bit platform. JSON places no bound on integer literals, so a corpus can carry integers that do not fit.

```
{ "id": 12345678901234567890 }
```

Options considered.

- (a) **Reject**, with an error naming the path and the offending value. Keeps the accepted fragment small at the cost of refusing corpora other tools accept.
- (b) **Promote the column to Int64**. Covers realistic 64-bit identifiers exactly; still bounded, so genuinely larger values remain an error.
- (c) **Promote the column to float**. ← **selected**
- (d) **Arbitrary precision via zarith (Z.t)**. Never lossy, but adds a dependency on GMP and makes the generated modules heavier to consume.

Decision. Promote to `float`. A path carrying any integer literal outside native `int` range becomes `float` for the whole column.

```
{ "id": 1 }  
{ "id": 12345678901234567890 }  
  
-> id : float (one out-of-range literal widens the column)
```

Rationale. This unifies S1b with S1a rather than adding a second mechanism. There is one widening lattice, $\text{int} \sqsubseteq \text{float}$, and two things push a column up it: a fractional literal, and an integer literal that does not fit. Inference stays monotone, and no new type appears in the generated code.

Implementation note. Overflow is detectable at parse time without custom lexing. Yojson’s `Safe` and `Raw` parsers report an out-of-range integer literal as `Intlit of string`, carrying the literal rather than failing; verified against yojson 3.0.0. Parsing with `Basic`, which lacks `Intlit`, would lose this distinction.

Known limitation. An IEEE double has a 53-bit significand, so integers above 2^{53} are rounded on conversion. A path of 64-bit identifiers will be silently altered, and two distinct identifiers can collapse to the same `float`. This is accepted deliberately; option (b) is the upgrade path and changes nothing else in this document.

S2. Strings

A path holding JSON strings.

```
{ "name": "tatami" }
```

Options considered.

- (a) **OCaml string**, holding the decoded UTF-8 bytes. ← **selected**
- (b) **A validated Unicode type**, rejecting or normalising ill-formed sequences at ingest.
- (c) **Refine string-encoded scalars** — recognise dates, UUIDs or numbers written as strings and give them narrower types.

Decision. OCaml `string`. No refinement, no validation beyond what the parser performs.

```
{ "name": "tatami" }

-> name : string
```

Rationale. Nothing in Phase 1 depends on a finer classification of strings. The one refinement the project does want — collapsing a string-encoded enumeration into an OCaml variant — is assigned to Phase 2 by the design note (§3.3), so introducing it here would duplicate that pass.

Known limitation. OCaml’s `string` is a byte sequence, so nothing guarantees the contents are well-formed UTF-8 beyond what `yojson`’s decoder enforces on escape sequences. Option (c) remains available later and is independent of every other decision here.

S3. Booleans

A path holding JSON `true` and `false`.

```
{ "active": true }
```

Options considered.

- (a) OCaml `bool`. ← **selected**
- (b) **A generated two-constructor variant**, for uniformity with whatever S18 chooses for paths carrying several types.

Decision. OCaml `bool`.

```
{ "active": true }
{ "active": false }

-> active : bool
```

Rationale. There is no choice to make here. A path that mixes booleans with values of another type is not this scenario; it is S18.

S4. Explicit null and absent fields

JSON distinguishes three states where OCaml’s `option` offers two: a key present holding a value, a key present holding `null`, and a key absent altogether.

```
{ "a": 1, "b": 2 } -- b present with a value
{ "a": 1, "b": null } -- b present, explicitly null
{ "a": 1 } -- b absent
```

Two independent facts are observable per path: whether the key is ever absent, and whether the value is ever `null`. Preserving both would require a three-state type; collapsing them fits `option` directly.

Options considered.

- (a) **Collapse.** `null` and absence both mean “no value”. A path that is ever absent or ever `null` becomes `option`, and `None` covers both. ← **selected**
- (b) **Keep them apart** in the type, as `int option option` or as a generated three-state variant type `'a field = Absent | Null | Value of 'a`. Exact, but heavier to consume and it propagates into the Phase 2 effect signatures.

- (c) **Distinguish in the schema only.** Record both facts during inference but generate a single `option`, so the distinction survives in the schema and the documentation but not in the type.
- (d) **Reject explicit null.** Only absence means “no value”; a corpus containing `null` is an error.

Decision. Collapse. A path whose key is absent in some document, or whose value is `null` in some document, becomes `option`; a path always present and never null carries no `option` at all.

```
{ "a": 1, "b": 2 }
{ "a": 1, "b": null }
{ "a": 1 }
```

-> a : int (always present, never null)
-> b : int option (None covers both null and absent)

A path needs only one document to make it partial; the rest may all supply values:

```
{ "x": 1 } { "x": 2 } { "x": 3 } { }
```

-> x : int option (absent once, so partial everywhere)

Rationale. This is the two-state framing the design note already uses when it says fields are classified as total or partial, and it is what comparable tools do. It keeps the generated modules directly consumable, and it keeps the Phase 2 effect signatures as simple as the underlying scalar allows.

Known limitation. The two absent-value cases become indistinguishable once shredded. A document carrying `"b": null` and a document omitting `b` produce identical rows, so the original cannot be recovered exactly: any round-trip property will hold only up to this collapse. A related consequence is that reconstruction must pick a canonical form when it emits a `None` — either `"b": null` or omitting the key — and that choice is not made here.

S6. Field never given a type

A path can be observed many times and still never supply a typed value: the key is present in every document, but its value is always `null`. The field plainly exists, yet nothing says whether it is an `int`, a `string`, or anything else.

```
{ "a": 1, "b": null }
{ "a": 2, "b": null }
```

Options considered.

- (a) **Reject**, with an error naming the path. Honest, but brittle: a single always-null field aborts an otherwise usable corpus, and real corpora carry such fields routinely.
- (b) **unit option**. The column exists, is always `None`, and its type states that no value was ever observed. ← **selected**
- (c) **Drop the field** from the schema entirely.
- (d) **An opaque raw-JSON column** (`Yojson.Safe.t`), leaving the typing to the consumer.

Decision. Emit `unit option`. The field is kept, and its type records that no value was ever observed at that path.

```
{ "a": 1, "b": null }
{ "a": 2, "b": null }

-> a : int
-> b : unit option (always None; no value was ever observed)
```

Rationale. Option (b) is the only one that records both facts that are true — the field exists, and its type is unknown — without rejecting the corpus. Option (c) is the one actively worth avoiding: dropping the field silently makes the generated modules disagree with the input about which fields exist, and does so with no diagnostic.

This also keeps S6 inside the same widening machinery as S1. The unknown type is the least element of the lattice, below every scalar type, and any observed value replaces it. `unit option` is simply how that least element is spelled in generated code, just as an out-of-range integer literal (S1b) is another way of moving up the same lattice rather than a separate mechanism.

Known limitation. The column carries no data. Under S4’s collapse it can only ever hold `None`, so it exists purely to record that the field was present in the input. The type is also not polymorphic: if a later corpus supplies a typed value at that path, the schema changes and the generated module changes with it, rather than the existing type being instantiated.

5 Objects

S7. Nested object, one-to-one

A field whose value is an object, with the same shape across the corpus. This is the worked example of the design note, §2.1.

```
{ "a": 1, "b": { "c": 1, "d": 2 } }
```

Options considered.

- (a) **Separate table with a foreign key.** The nested object becomes its own table and module; the parent holds a foreign-key column and gains an accessor that follows it. ← **selected**
- (b) **Flatten into the parent** with prefixed columns (`b_c`, `b_d`). No child module and no join. This is what Prisma and Airbyte’s basic normalization do for one-to-one nesting.
- (c) **An inline record type** in the parent module, with no separate table and no identifier.
- (d) **Hybrid** — flatten when the nested object is total, split it into a table when it is nullable.

Decision. Separate table with a foreign key. Every nested object becomes its own table and its own module, the parent carries a foreign-key column holding the child’s `id`, and the parent module exposes an accessor that follows it.

The two tables that come out, with the identifier columns that shredding invents:

Table root			Table b		
id	a	b_id	id	c	d
1	1	1	1	1	2

and the modules they generate:

```

module B : sig
  type id
  type t = { id : id; c : int; d : int }
  val get : db -> id -> t
end

module Root : sig
  type id
  type t = { id : id; a : int; b_id : B.id }
  val get : db -> id -> t
  val b : db -> t -> B.t (* follows the foreign key into table b *)
end

```

Under S4, a nested object that is sometimes absent or null makes the foreign-key column partial, and the accessor partial with it:

```

{ "a": 1, "b": { "c": 1, "d": 2 } }
{ "a": 2 }

```

```

type t = { id : id; a : int; b_id : B.id option }
val b : db -> t -> B.t option

```

Rationale. This is what the design note specifies — nested and repeated substructures are normalized into separate relations linked by generated keys, with foreign-key constraints materialized rather than left implicit — and what its §2.1 code emits. It is also what Phase 2 needs: the whole effect lowering is a rewrite of these accessors into generators, and a flattened field would present nothing to rewrite.

Known limitation. Every nested field costs a join. Option (b) avoids it, and matters for Phase 3 where a flattened column can be reached without one; option (d) remains available later as an optimization for the total case, since it changes only how a total nested object is realized and nothing else in this document. Note also that flattening degrades badly under S4: if the nested object is sometimes absent, every flattened column becomes optional and `{"b": {"c": null, "d": null}}` is no longer distinguishable from `{}`, whereas a single nullable foreign key records exactly that fact.

Implementation note. The child table takes its name from the field, so two different parents each holding a field named `b` would collide. Naming and collision resolution are S22. Note also that the generated signature uses `b` both as a record field and as a value; OCaml keeps record fields and values in separate namespaces so this compiles, but it reads poorly and S22 may choose to disambiguate.

S8. Empty object

A field whose value is an object with no members, at every document in the corpus. If some documents supply members and others do not, ordinary inference applies and the members are simply partial under S4; this scenario is only the case where the object is empty everywhere.

```

{ "a": 1, "b": {} }

```

Options considered.

- (a) **A table with only an identifier column**, and a module whose row type carries nothing but that identifier. ← **selected**
- (b) **Reject**, with an error naming the path.

(c) **Drop the field** from the schema.

(d) `b : unit` — a marker in the parent, with no table and no module.

Decision. Emit a table with only an identifier column. The parent holds a foreign key to it exactly as in S7.

```
module B : sig
  type id
  type t = { id : id }
  val get : db -> id -> t
end

module Root : sig
  type id
  type t = { id : id; a : int; b_id : B.id }
  val get : db -> id -> t
  val b : db -> t -> B.t
end
```

Rationale. The same principle as S6: keep the field, and let its generated form state exactly what is known about it. Option (a) also preserves a distinction the others lose. `{"b": {}}` and `{}` are different inputs: the first yields a non-null foreign key to an empty row, while the second makes `b` partial under S4. Under (c) and (d) that difference disappears.

Option (d) is the tempting shortcut, and it holds only until some document in the corpus gives `b` a member — at which point the representation has to change from `unit` to a module, rather than the existing table merely gaining a column.

Known limitation. The table carries no data. Like the `unit` option of S6, it exists to record structure that was present in the input, and every row in it is a bare generated identifier.

S9. Map-shaped object (keys are data)

Two inputs that are structurally identical and semantically unrelated:

```
{ "point": { "x": 1, "y": 2 } }

{ "users": { "u_1001": { "name": "ada" },
              "u_1002": { "name": "grace" } } }
```

Both are objects whose members are values. But `x` and `y` are *schema*, repeated in every document, while `u_1001` is *data*, and the next document carries entirely different keys. Nothing in the JSON distinguishes them.

Why this decides more than it appears to. The property worth protecting is that **the schema is a function of document structure, not of data values**. Treating a map as a record violates it: the generated record's fields become the identifiers that happened to appear in the sample, so a document arriving later with a new key has no column to go in.

```
inference corpus: { "users": { "u_1001": {...}, "u_1002": {...} } }
                  -> record with fields u_1001, u_1002

later document: { "users": { "u_1003": {...} } }
                 -> no such column; shredding is undefined here
```

For a statement quantified over the exact corpus that was used for inference, this is harmless. For the statement that matters — schema fixed once, documents arriving afterwards —

shredding must be total on everything the schema accepts, and here it is not: a new key is a schema change and a recompilation rather than a new row.

Options considered.

- (a) **No detection.** Every object is a record. Deterministic, but makes the schema depend on data values, as above.
- (b) **Heuristic detection**, lowering automatically as a map on signals such as key-set variability, key count, and uniformity of the values. Makes the schema a function of the sample: adding a document can flip a path between record and map.
- (c) **Explicit, path-directed marking.** Every object is a record unless its path is marked as a map. $\leftarrow$ **selected**
- (d) **Heuristic detection that rejects** rather than guesses, directing the user to mark the path.

Decision. Explicit, path-directed marking. Inference takes the marking as an additional input, so the signature becomes $Corpus \times Annotation \rightarrow Schema$ — still deterministic, still total, and still a function of structure once the marking is given.

A marked path lowers to the same table an array of objects produces, with the ordering index replaced by a string key. The parent gains no column; the child carries the parent reference and the key, and since the record is the row (S21) both appear as fields:

```
module User : sig
  type id
  type t = { id : id; root_id : Root.id; key : string; name : string }
  val get : db -> id -> t
end

module Root : sig
  type id
  type t = { id : id }
  val get : db -> id -> t
  val users : db -> t -> User.t list
end
```

Surface form. The marking is written as a configuration file listing paths:

```
maps = [".users", ".index.by_id"]
```

This is the form the web interface exports, so the same file drives generation both inside and outside the interface and a generated result can be reproduced without it. An inline annotation or an interface toggle would be interchangeable — each identifies *paths* — but the file is the single form of record.

A configuration option that names a *threshold* — treat any object with more than N members as a map — is not an equivalent spelling and is excluded. It is deterministic given N , but it remains a function of data values: member counts grow with the corpus, so adding documents can carry a path across the threshold and change the schema. Only path-directed marking preserves the structural property above.

Rationale. This keeps shredding total on documents that arrive after the schema is fixed — a new key is a new row, not a new column — and it keeps the eventual equivalence statement free of any need to quantify over an unspecified heuristic. Formally the marking is an input, not a guess.

Known limitation. Determinism is not protection. An unmarked map still lowers as a record and still produces a module with one field per key. A detection heuristic may be added later as a usability aid, but it must remain advisory: it may surface a suggestion for a person to accept, and must never change what is generated on its own.

S10. Duplicate keys

An object carrying the same key more than once.

```
{ "a": 1, "a": 2 }
```

RFC 8259 says member names *should* be unique and declares the behaviour undefined when they are not. Implementations genuinely disagree: some keep the last binding, some the first, some raise an error.

Options considered.

- (a) **Reject**, with an error naming the path and the repeated key. ← **selected**
- (b) **Last binding wins**, as JavaScript's `JSON.parse` does.
- (c) **First binding wins**.
- (d) **Treat the key as repeated**, lowering the path to a collection as though the document had supplied an array.

Decision. Reject. A document containing a repeated key within any object, at any depth, is an error naming the path and the key.

Rationale. The goal is agreement between querying the JSON and querying the database built from it. Here the JSON side has no defined answer: two conforming JSON query engines may legitimately disagree about the value of `.a` in the document above. Agreement cannot be proved against a reference semantics that is itself undefined, so admitting duplicates would leave a hole in the statement at precisely the point where the statement is the product.

Options (b) and (c) discard data silently. Option (d) preserves it but invents a collection the document did not ask for, and makes the schema depend on whether a key happened to repeat. Rejection also costs little in practice: duplicates are rare, and when they occur they usually indicate a fault in whatever generated the document — something the user would rather be told about than have quietly resolved.

Implementation note. The parser will not perform this check for us. Yojson represents an object as an association list and preserves both members: `{"a":1, "a":2}` parses to a two-element list and re-serializes with both, verified against yojson 3.0.0. The check must therefore be explicit, and applied to every object at every depth. It is a per-document check, not a corpus-level one.

Known limitation. Documents that other tools accept will be rejected. Since the check is per-document, one malformed document rejects the whole input unless the interface offers to skip it.

S11. Key names that are not OCaml identifiers

JSON permits any string as a member name; OCaml record fields must be lowercase identifiers drawn from a restricted ASCII set and must not collide with keywords. Several distinct failures arise:

```
{ "my-field": 1, -- hyphen is illegal in an identifier
  "type": 2, -- an OCaml keyword
  "1abc": 3, -- leading digit
  "Name": 4, -- leading uppercase; record fields must start lowercase
  "caf\u00e9": 5, -- non-ASCII; OCaml identifiers are ASCII only
  "": 6 } -- empty
```

The capitalized case is the one most easily overlooked: a capitalized key is entirely ordinary JSON, emitted by most .NET and Java producers, and cannot be a record field name at all.

The binding constraint is **injectivity**. Two distinct JSON keys must never map to the same OCaml name, or two columns silently become one. Naive replacement fails at once: `my-field` and `my_field` both become `my_field`.

Options considered.

- (a) **Unconditional injective mangling**. A total scheme applied to every key, with the original key retained in the schema. Names are stable and always distinct, and frequently ugly. ← **selected**
- (b) **Reject** any key that is not already a legal OCaml identifier.
- (c) **Context-sensitive mangling** — use the readable form and escape only when that particular object would otherwise collide.
- (d) **Naive mangling with collision detection** — simple replacement, erroring when two keys collide.

Decision. Unconditional injective mangling. Every key is mangled by the same total function, whether or not it is already legal, and the original key is recorded in the schema alongside the mangled name.

Implementation note. A concrete scheme, sufficient to implement and adjustable in its cosmetic details so long as injectivity is preserved:

1. Process the key as UTF-8 bytes. A byte in `a-z`, `A-Z`, `0-9` or `'` passes through unchanged; every other byte, *including* `_`, becomes `_` followed by two lowercase hexadecimal digits. Escaping `_` is what makes `_` an unambiguous escape introducer.
2. If the result is empty or does not begin with a lowercase letter, prefix it with `f_`.
3. If the result is an OCaml keyword, append `_`.

Under this scheme `my-field` becomes `my_2dfield`, `type` becomes `type_`, `1abc` becomes `f_1abc`, and `Name` becomes `f_Name`.

That this scheme is injective is a property to be checked rather than assumed; it is exactly the kind of small lemma worth proving once the formal development resumes.

Rationale. Option (b) rejects a large fraction of real-world JSON: hyphenated and capitalized keys are commonplace, and the user usually cannot rename the source data. Option (c) makes a generated name depend on what else appears in the same object, so adding a key to the corpus can rename an existing field and break code written against the generated module — the same data-dependence problem as S9, in a milder form, since an object's key set is structure rather than data.

Reconstruction does not require the mangling to be invertible by inspection. The original key is stored in the schema, so injectivity alone is what is needed: it prevents two columns from merging, and the stored key restores the exact name on output.

Known limitation. Mangled names are less readable than the keys they came from, and the delivery vehicle sharpens the cost: these modules are read by people in an editor rather than consumed as a build artifact. A middle path was considered — mangle readably but reserve the escaped forms, so collisions are impossible by construction — which recovers most of (c)’s readability while retaining (a)’s stability. It remains available as a later refinement.

Module names, which must instead begin with an uppercase letter, need a separate rule; that belongs to S22 along with collisions between generated names.

6 Arrays

S13. Array of objects

A field whose value is an array of objects.

```
{ "id": 1, "items": [ { "sku": "a", "qty": 2 },
                     { "sku": "b", "qty": 1 } ] }
```

Options considered.

- (a) **Child table with a back-reference and an ordering index**, and a presence marker in the parent when the path is partial. ← **selected**
- (b) **Child table with a back-reference but no index**. The array becomes a set rather than a sequence.
- (c) **Parent holds a list of child identifiers**, keeping the direction of S7.
- (d) **A junction table** between parent and child, as for a many-to-many relationship.
- (e) **Parent references a collection entity**, with the elements referencing the same collection: `root(id, items_id), item_list(id), items(id, list_id, idx, ...)`.

Decision. Child table carrying a foreign key back to the parent and an ordering index. The parent gains no column when the path is total, and a presence marker when it is partial.

Table root	Table items				
id	id	root_id	idx	sku	qty
1	1	1	0	a	2
	2	1	1	b	1

```
module Item : sig
  type id
  type t = { id : id; root_id : Root.id; idx : int;
            sku : string; qty : int }
  val get : db -> id -> t
end

module Root : sig
  type id
  type t = { id : id }
  val get : db -> id -> t
  val items : db -> t -> Item.t list
end
```

The presence marker. An empty array is a *present* value, so it must not be confused with an absent field. Both produce zero rows in the child table, and the parent is therefore where the difference has to be recorded. This is not new machinery: S4 already makes a path that is sometimes absent partial, and producing an *option* at all requires the parent to record presence.

```
{ "id": 1, "items": [ { "sku": "a" } ] }
{ "id": 2, "items": [] }
{ "id": 3 }
```

```
type t = { id : id; items_present : bool }
val items : db -> t -> Item.t list option

root 1 -> Some [ {sku = "a"} ]
root 2 -> Some [] (* present, empty *)
root 3 -> None (* absent *)
```

Under S4, "items": null collapses with the absent case and yields *None*. The marker also appears as a record field, since the record is the row (S21.3).

Rationale. The ordering index is required rather than optional. JSON array order is significant, so without it ["a","b"] and ["b","a"] shred identically, reconstruction recovers a multiset rather than a list, and a query asking for the first element has an answer on the JSON side that the database cannot reproduce. Option (b) is excluded on exactly that ground.

Option (c) is not relational — a list-valued column is not a column — and would still need an index to order it. Option (e) was considered seriously, since it is more uniform: parent-holds-reference would then be the single rule for both nested objects and arrays, instead of the direction flipping with cardinality. It was declined because, absent any sharing of arrays between documents, *item_list* holds exactly one row per parent row. It carries no information beyond “this array exists”, which is what the presence marker records directly, and it costs an extra join on every access. Option (e) becomes worthwhile only if arrays are given identity so that equal arrays can be shared, which would make shredding depend on structural hashing.

Known limitation. A normalization treating [], null and absence as equivalent at array-typed paths was considered. It would remove the presence marker entirely and make array accessors total, at the cost of stating the equivalence in the specification and accepting that the resulting theorem is relative to that semantics rather than to the ones implemented by jq, JSONPath or SQL/JSON, all of which distinguish null from []. It is recorded here as an available simplification, not adopted.

As with nullability generally, the parent table’s shape depends on what was observed: a path that is total in the corpus has no presence marker, and one that is partial has one.

S14. Array of scalars

A field whose value is an array of scalars rather than of objects.

```
{ "id": 1, "tags": ["red", "blue"] }
```

Relationally this is S13 with a scalar element in place of an object: a child table carrying a foreign key to the parent, an ordering index, and a single value column. What is open is the module surface, since a record wrapping one string is an awkward thing to hand a consumer.

Options considered.

- (a) **A full child module**, exactly symmetric with S13. Uniform, but the consumer unwraps a one-field record to reach a string.

- (b) **Table without a module.** The accessor returns the scalars directly and no child module is generated. ← **selected**
- (c) **Both** — generate the module, and add a convenience accessor returning the scalars.
- (d) **Reject** arrays of scalars, requiring them to be wrapped as objects.

Decision. The child table exists relationally, but generates no module. The parent exposes a single accessor returning the scalars themselves.

Table root	Table tags			
id	id	root_id	idx	value
1	1	1	0	red
	2	1	1	blue

```
module Root : sig
  type id
  type t = { id : id }
  val get : db -> id -> t
  val all : db -> t list
  val tags : db -> t -> string list
end
```

Presence. The presence marker of S13 applies unchanged. A partial path yields `val tags : db -> t -> string list option`, with the parent record carrying the presence marker of S21.3, and `Some []` for a present empty array and `None` for an absent or null one.

Rationale. One construct rather than two. The consumer receives the `string list` they actually want, with no one-field record to unwrap and no second accessor to choose between, and the specification has a single accessor per scalar-array path to describe rather than a pair whose relationship must also be stated.

The relational layer is untouched: the table, its foreign key and its ordering index all exist exactly as in S13. What is dropped is only the module that would have wrapped them.

Known limitation. This breaks the design note’s table-to-module correspondence: a table exists with no module, so the correspondence holds in one direction only. The rows of that table are also not addressable — there is no `Tags.id` and no way to name an individual element — so if element identity is ever needed, option (c) is the way back, and adopting it would be additive rather than breaking.

S15. Empty arrays

A path that is an array in every document, but where no document ever supplies an element, so no element type is ever observed. If some document does supply elements, ordinary inference applies and this scenario does not arise: the element type comes from wherever it was seen.

```
{ "id": 1, "tags": [] }
{ "id": 2, "tags": [] }
```

Options considered.

- (a) **Reject**, with an error naming the path.
- (b) **A child table with an identifier, a parent reference and an index, but no value column.** The array exists and is always empty. ← **selected**

(c) Drop the field.

(d) **An opaque raw-JSON element column**, deferring the element type to the consumer.

Decision. Emit the child table with no value column, and follow S14: the table generates no module, and the parent exposes a single accessor.

Table root	Table tags
id	id root_id idx
1	(no rows)

```
module Root : sig
  type id
  type t = { id : id }
  val get : db -> id -> t
  val all : db -> t list
  val tags : db -> t -> unit list
end
```

Rationale. The same principle as S6: keep the field, and let its generated form state exactly what is known — here, that the path is an array and that its element type was never observed. This is again the least element of the type lattice, and if a later corpus supplies an element the value column appears and the element type is inferred, exactly as an always-null scalar path would gain a type.

Known limitation. The table is more degenerate than S6’s always-None column: it has no value column and, in a corpus of this shape, no rows either. It nonetheless carries one bit of information, because the presence marker retained in S13 still distinguishes `{"tags": []}` from `{}` — Some `[]` against None — so the table records that the field was present even though it never records a value.

S16. Heterogeneous array

A single array whose elements have types with no common supertype.

```
{ "id": 1, "mixed": [ 1, "two", { "three": 3 } ] }
```

This is the first scenario where the widening used everywhere above stops working. S1 and S6 widen along a chain — $\perp \sqsubseteq \text{int} \sqsubseteq \text{float}$ — so any two observations have a least upper bound. An integer, a string and an object have none, so there is nothing to widen to.

Options considered.

- (a) **Reject**, with an error naming the path and the conflicting types. ← **selected**
- (b) **A generated variant.** The child table becomes a sparse table — `mixed(id, parent_id, idx, tag, v_int, v_str, v_obj_id)` with exactly one value column non-null per row — and the element type becomes `type mixed_elt = Int of int | Str of string | Obj of Mixed_obj.id`.
- (c) **An opaque raw-JSON column**, leaving elements as `Yojson.Safe.t` for the consumer to interpret.
- (d) **Widen to string**, serializing non-string elements back to JSON text.

Decision. Reject. An array whose elements have no least upper bound is an error naming the path and the types observed.

What does not count as heterogeneous. The rejection is narrow, and several mixtures that look heterogeneous are resolved by decisions already taken:

- `[1, 2.5]` — integers and floats widen to `float` under S1a. Not a conflict.
- `[{"a":1}, {"b":2}]` — objects with different members merge into one table whose columns are partial under S4. Not a conflict.
- `[[1], [2]]` — arrays of arrays are S17, not this scenario.

What is rejected is a genuine absence of a least upper bound: integer against string, integer against boolean, string against boolean, or any scalar against an object or an array.

Rationale. Option (b) is the principled answer and the one a relational modeller would reach for, but it reintroduces the data-dependence problem of S9 in a third costume: the *shape of the generated type* would depend on which element types happened to appear, so adding a document could add a constructor and change the generated variant. Option (d) discards the type information it claims to preserve, and option (c) defers the whole problem to the consumer while making the column untyped.

Rejection also matches what such arrays usually are in practice. A corpus mixing scalars and objects at one path generally reflects a fault in whatever produced it, and is something the user would rather be told about.

Known limitation. Option (b) remains the natural extension if heterogeneous arrays turn out to matter, and adopting it later would not disturb any other decision in this document. Note that S18 asks the same question across documents rather than within an array, and the two answers should be kept consistent.

S16b. Null elements in an array

S4 governs object members rather than array elements, so it does not by itself decide the status of a null sitting in element position.

```
{ "mixed": [ 1, null, 2 ] }
```

Options considered.

- (a) **null is the absence of a value.** The element column becomes nullable, the element type is `int option`, and the array is not heterogeneous. ← **selected**
- (b) **null is a distinct type** with no least upper bound against `int`, making the array heterogeneous and rejecting it under S16.

Decision. Treat `null` in element position as the absence of a value. The element column is nullable and the element type gains an `option`; the array is not heterogeneous.

```
{ "mixed": [ 1, null, 2 ] }  
-> element type int option; the value column is nullable  
  
{ "mixed": [ null, null ] }  
-> element type never observed; S15 applies, with a nullable column
```

Rationale. Consistent with the rest of this document, where `null` has uniformly meant the absence of a value rather than a type of its own. S6 already gave an all-null path a home rather than treating it as a conflict, and rejecting `[1, null, 2]` while accepting an all-null scalar field would be hard to justify.

The contrary reading was weighed: an array has no notion of a missing element, so `[1, null, 2]` has three elements and the middle one is a value that happens to be null, which

makes `null` more type-like in element position than in member position. It is recorded here and not adopted.

A collapse that does not occur. Unlike S4, this introduces no loss. In member position `None` is ambiguous between an explicit `null` and an absent key. In element position there is no absent case to collide with — every index up to the array’s length is occupied — so a `null` element reconstructs as `null` unambiguously. Element nullability is lossless where member nullability is not.

S17. Array of arrays

An array whose elements are themselves arrays.

```
{ "id": 1, "matrix": [ [1, 2], [3] ] }
```

The outer array is handled by S13, but its elements are neither scalars nor objects, and an array has no relational form of its own. The element has nowhere to go.

Options considered.

- (a) **Reject**, with an error naming the path. ← **selected**
- (b) **One table per level.** The outer array’s table holds an identifier and an index; the inner array’s table references it and carries its own index and the value. Two tables and two index columns, generalising to arbitrary depth.
- (c) **A single table with one index column per level** — `matrix(id, parent_id, idx0, idx1, value)`. Flatter, but the number of index columns then depends on the nesting depth found in the data.
- (d) **An opaque raw-JSON column** holding the inner arrays.

Decision. Reject. An array whose elements are arrays is an error naming the path.

What is rejected is narrow. Only a *direct* array-inside-array is refused. An array nested inside an object which is itself inside an array is ordinary and remains accepted, because each level is then mediated by a named member:

```
{ "matrix": [ [1, 2], [3] ] } -- rejected  
  
{ "matrix": [ { "row": [1, 2] }, { "row": [3] } ] } -- accepted
```

The second form is the escape hatch for data of this shape, and it costs the user one wrapper object per level.

Rationale. The decisive objection is naming. Every table takes its name from the field whose value it holds, but an inner array sits in *element* position, which has no key. There is nothing to name its table after, so option (b) does not merely generate an ugly module — it generates one that nothing can sensibly name, and it generates a fresh one for every level of nesting. Since arrays may nest to any depth, that is an unbounded family of anonymous tables, each contributing a join.

Requiring the wrapper object restores a name at every level, which is why the accepted form above is not an arbitrary restriction: it asks the document to supply the one piece of information the relational form needs and the bare nested array withholds.

Option (c) additionally makes the table’s shape depend on the nesting depth observed in the data, which is the data-dependence problem of S9 once more.

Known limitation. Legitimate data is rejected — matrices and other numeric structures are naturally written as nested arrays, and their producers cannot usually be asked to insert wrapper objects. Option (b) remains the natural extension if this proves too restrictive, and adopting it would not disturb any other decision here, though it would first require S22 to settle how an unnamed table is named.

7 Corpus-level scenarios

S18. Field with differing types across documents

A path whose observed values have types with no least upper bound, where the conflicting observations come from different documents.

```
{ "id": 1, "value": 42 }
{ "id": 2, "value": "unknown" }
```

Options considered.

- (a) **Reject**, with an error naming the path and the conflicting types. ← **selected**
- (b) **A generated variant** — `type value_ty = Int of int | Str of string` — with the column becoming a tag plus one nullable column per alternative.
- (c) **An opaque raw-JSON column.**
- (d) **Widen to string**, serializing non-string values back to JSON text.

Decision. Reject, consistently with S16.

S16 and S18 are one rule. Under the path-based framing of this document they are not two scenarios but one, stated at two kinds of path. All elements of an array share the path `.mixed[]`, and all values of a member share the path `.value`; in both cases every observation across the whole input feeds that path, and the requirement is the same:

The types observed at a path must have a least upper bound in the type lattice.

S16 is this rule at an element path, S18 the same rule at a member path.

What does not count as a conflict. As in S16, the rejection is narrow. Every mixture below is resolved by a decision already taken and is accepted:

- `int` and `float` widen to `float` (S1a), as does an integer literal outside native range (S1b).
- Any type together with `null` or with the key being absent becomes an `option` (S4).
- A path never given a type sits below every type and is replaced by any observation (S6).
- Objects with different members merge into one table whose columns are partial (S4).

What is rejected is a genuine absence of an upper bound: integer against string, integer against boolean, string against boolean, or any scalar against an object or an array.

Rationale. The arguments from S16 carry over unchanged. Option (b) would make the shape of a generated *type* depend on which values happened to appear, so that adding a document could add a constructor — the data-dependence problem of S9 again. Option (d) discards the type information it claims to preserve, and option (c) leaves the column untyped. A corpus that

does this at a path usually reflects a producer that changed its mind rather than a deliberate union.

Known limitation. This is the one decision in the document that breaks monotonicity. Everywhere else, adding documents can only loosen an inferred type: a column widens from `int` to `float`, or from total to partial, and a schema inferred from a smaller corpus stays valid. Rejection under this rule can instead invalidate a corpus that worked before, because one new document can introduce a conflict at a path that was previously uniform.

The scenario is also invisible under single-document input (S0), since it takes two documents to produce, and the diagnostic must therefore name a pair of documents and the type each contributed rather than pointing at one place in one file. Option (b) remains the extension if unions turn out to be worth supporting, and it would then have to be adopted for S16 at the same time.

S19. Recursive and mutually recursive shapes

A corpus whose structure refers to itself: trees, comment threads, file systems, organisation charts.

```
{ "value": 1,
  "children": [ { "value": 2, "children": [] },
                { "value": 3, "children": [ { "value": 4,
                                              "children": [] } ] } ] }
```

Doing nothing is not rejection. JSON documents are finite trees, so any corpus has bounded depth and naive inference emits one table per level observed — `node`, `node_children`, `node_children_children`. Nothing fails. But the number of tables then depends on how deep the sample happened to go, and a document arriving later that is deeper than the sample has nowhere to be put: shredding becomes partial. This is the same defect that ruled out treating maps as records in S9.

Self-recursion and mutual recursion differ in cost. A single self-referential table needs no special machinery: `val children : t -> t list` refers to `t` within its own signature, which is ordinary OCaml. Recursion running through two or more tables requires `module rec`, which carries real restrictions.

Options considered.

- (a) **Detect and reject.** Note this still requires the detection machinery of (b); it merely errors rather than using it.
- (b) **Detect and emit a recursive schema.**
- (c) **No detection** — one table per observed depth, as above.
- (d) **Explicit annotation**, as in S9: the configuration names the recursive paths. ← **selected**

Decision. Explicit annotation. The configuration file identifies a path with an ancestor path it folds into; inference then unifies the two rather than generating a fresh table per level.

```
recursive = [ { path = ".children[]", folds_into = "." } ]
```

The annotation identifies a *pair* of paths, unlike S9's marking which names a single path. Naming only the recursive path would require inferring which ancestor it folds into, reintroducing the detection this option exists to avoid.

The two paths unify into one table with a self-referencing foreign key. Root rows have no parent, so the parent reference and the index are both partial:

Table node			
id	parent_id	idx	value
1	—	—	1
2	1	0	2
3	1	1	3
4	3	0	4

```
module Node : sig
  type id
  type t = { id : id; parent_id : id option; idx : int option;
             value : int }
  val get : db -> id -> t
  val children : db -> t -> t list
end
```

Where the recursion runs through more than one table, the generated modules become mutually recursive:

```
module rec Post : sig
  type id
  type t = { id : id; title : string }
  val get : db -> id -> t
  val comments : db -> t -> Comment.t list
end
and Comment : sig
  type id
  type t = { id : id; post_id : Post.id; idx : int; body : string }
  val get : db -> id -> t
  val post : db -> t -> Post.t
end
```

The annotation is validated, not trusted. After inference, the two paths named `must` actually have compatible shapes. If they do not, the annotation is an error naming both paths and the mismatch. Otherwise a mistaken annotation would produce a schema the data does not fit.

Rationale. Detection is sample-dependent, which is what rules out (b) and (a). Whether `.children[]` has the same inferred shape as the root depends on what was observed: if no child in the sample has grandchildren, then `.children[].children` was only ever empty, its element type is unknown under S15, and it need not compare equal to the root’s shape. The schema would then be recursive or not according to how deep the corpus happened to go — the data-dependence problem of S9.

Annotation avoids this the same way S9 does, and reuses the configuration mechanism already introduced there. It also keeps shredding total: once a path is known to be recursive, a document deeper than anything in the corpus still has a table to go in.

Known limitation. As in S9, this gives determinism rather than protection. Recursive data that is not annotated still lowers as one table per observed depth, and is still partial on deeper documents; the user only finds out by inspecting the generated modules. A detector could be added later as an advisory hint, subject to the same constraint as S9’s: it may suggest an annotation for a person to accept, and must never change what is generated on its own.

S20. Required versus optional inference

S4 fixed how a partial path is encoded. This scenario asks when a path counts as partial in the first place.

```
{ "a": 1, "b": 2 } x999
{ "a": 1 } x1
```

Options considered.

- (a) **Any absence makes the path partial.** One document out of a thousand missing `b` gives `b : int option`.
- (b) **A threshold** — partial only when absent in more than some fraction of documents.
- (c) **Any absence makes the path partial, and the observation counts are reported** so that a single anomalous document can be found. ← **selected**
- (d) **The user declares it** in the configuration, with the corpus checked against the declaration.

Decision. A path is partial exactly when it is absent or null in at least one observation. Inference additionally records, and the interface reports, how many observations of each kind were seen at every path. The counts are diagnostic only and never affect what is generated.

```
.a present with a value 1000 / 1000 -> a : int
.b present with a value 999 / 1000
  absent 1 / 1000 -> b : int option
```

The report is the natural place for what S4 discards. S4 collapses `null` with absence, so the generated type cannot say which occurred. The diagnostic can, and should: reporting present-with-value, explicitly-null and absent as three separate counts surfaces exactly the information the collapse removes from the type, at no semantic cost.

Denominators differ by depth. The counts are per path, and a path’s denominator is the number of *observations* at that path, not the number of documents. For a root member that is one per document; for `.items[].qty` it is one per array element across the whole input. A ratio of 999/1000 therefore means different things at different depths, and the report should say what it is counting.

Rationale. Option (b) is a threshold on data values, which is the shape of rule excluded in S9: a field would flip between `int` and `int option` as the corpus grew, making the generated type depend on how much data happened to be supplied. Option (c) keeps generation purely structural — it is (a) semantically, and the generated code is identical — while supplying the information a user needs to notice that a field was missing exactly once and go and look at that document. Given that the tool is used through an interface where the schema is inspected before the modules are taken away, that information is worth surfacing next to each field.

Known limitation. A single malformed document makes an otherwise total field optional for every consumer of the generated module, and the report says so without fixing it. The only remedy is to remove the document from the input.

The counts are metadata rather than schema, so if they are to be useful outside the interface they must be exported alongside it, in the same way the configuration of S9 and S19 must be.

8 The generated module surface

Two principles govern this section, and between them they settle most of it.

The record is the row. A module’s record type has exactly the columns of its table — no hidden columns and no projections — so that $\llbracket M.t \rrbracket$ is the row type of M ’s table. Accessors are derived navigation on top of it.

The store is explicit. Every function that reads rows takes the database as its first argument, so every generated function is a function of its arguments.

The generated interface therefore opens with an abstract handle:

```
type db
```

How a `db` is obtained is a question about storage rather than about lowering, and is not specified here.

S21.1. Abstractness of `id`

How much of the identifier type the generated signature exposes.

Options considered.

- (a) **Fully abstract** — `type id`, with no operations. The only thing a consumer can do with an identifier is pass it back to `get`. ← **selected**
- (b) **Abstract with an operations bundle** — `equal`, `compare`, `to_string`, `hash`.
- (c) `type id = private int`, which coerces freely to `int` but cannot be forged.
- (d) `type id = int`, transparent.

Decision. Fully abstract, as written in the design note.

```
module Root : sig
  type id
  type t = { id : id; a : int }
  val get : db -> id -> t
end
```

Rationale. This makes `Root.id` and `B.id` distinct types, so an identifier belonging to one table cannot be passed where another’s is expected — an easy mistake to make when every identifier is an integer underneath. It is also faithful to the design note’s framing of these columns as generated rather than part of the document.

Known limitation. A consumer cannot print an identifier, compare two, hash one, or use one as a map key. Both (b) and (c) remain available, and adopting either later is backward compatible: (b) only adds functions, and (c) only weakens an abstract type to a private abbreviation.

S21.2. The store behind `get`

Options considered.

- (a) **An implicit ambient store**, as the design note writes it.
- (b) **An explicit handle** — `val get : db -> id -> t`. ← **selected**

(c) **A functor over a storage signature.**

(d) **The store is the effect handler**, supplying rows in response to a performed effect.

Decision. An explicit handle. Every function that reads rows takes a `db` as its first argument.

Rationale. Under an ambient store no generated function is a function of its arguments: `get : id -> t` reads hidden mutable state, two calls need not agree, and the signature does not admit that it depends on anything but its identifier. Threading the handle makes each generated function pure and its denotation immediate — $\llbracket \text{get} \rrbracket d\ i$ is the row named by i in d — which is what any equivalence statement about these modules has to quantify over.

This diverges from the design note, which writes the ambient form. The divergence is deliberate and is the price of a specification that can be stated without an implicit parameter running through every clause.

get is total. It requires no `option` and cannot fail. Identifiers are abstract (S21.1), so the only way to obtain one is from a module that already holds it; and foreign-key constraints are materialized, so every identifier in circulation names a row that exists. Totality follows from those two decisions together rather than being assumed.

Known limitation. Every accessor gains a parameter, which makes navigation chains noisier to write: `B.c d (Root.b d r)` rather than `B.c (Root.b r)`. Option (d) remains the eventual destination if Phase 2’s handlers come to supply rows, at which point the handle could be dropped again.

S21.3. The presence marker in the record type

S13 records presence in the parent when an array-valued path is partial, so that the accessor can return an `option`. This asks whether that marker is visible in the record.

Options considered.

(a) **Only through the accessor.** The record holds nothing.

(b) **In the record as well**, as a field mirroring the column. ← **selected**

(c) **In the record only**, with the accessor returning a plain list.

Decision. The marker appears in the record, because it is a column and the record is the row.

```
module Root : sig
  type id
  type t = { id : id; items_present : bool }
  val get : db -> id -> t
  val items : db -> t -> Item.t list option
end
```

Rationale. Follows directly from the governing principle. Keeping the record in exact correspondence with the table means the generated type needs no separate description: it is the row, and nothing has to be said about which columns were withheld.

Known limitation. A consumer now has two representations of one fact — the `items_present` field and the accessor’s `option` — which can be read inconsistently, and `items_present : bool` beside an `option`-returning accessor is the least attractive consequence of the principle. Option (c) is excluded regardless: it withholds the `option` and moves the obligation to check onto the caller.

S21.4. Foreign key field naming

A foreign key appears twice: as the record field holding the child’s identifier, and as the accessor that follows it. Naming both after the JSON member makes `r.b` and `b d r` look nearly identical while having different types.

Options considered.

- (a) **Both named `b`**, as the design note’s code writes it.
- (b) **Field `b_id`, accessor `b`** — matching the relational column. ← **selected**
- (c) **Field `b`, accessor renamed to `get_b`**.
- (d) **Hide the foreign key from the record.**

Decision. The field is named for the column, `b_id`; the accessor keeps the member’s name.

```
module Root : sig
  type id
  type t = { id : id; a : int; b_id : B.id }
  val get : db -> id -> t
  val b : db -> t -> B.t (* follows b_id into table b *)
end
```

Rationale. This follows from the governing principle rather than being an independent choice: the column is named `b_id`, so the field is too. It removes the clash for free.

The design note is not internally consistent here — its relational table names the column `b_id` while its generated record names the field `b` — so this decision follows the note’s table where S21.1 followed its code. Option (d) is excluded by the principle, since a foreign key is a column.

S21.5. Entry point into the modules

Every accessor starts from a value of type `t`, and `get` requires an identifier obtainable only from a module already holding one. Nothing in the design note produces the first row.

Options considered.

- (a) `val all : db -> t list` on every generated module. ← **selected**
- (b) `val all : db -> t Seq.t` — lazy, so the table is not materialized.
- (c) **An entry point on root tables only.**
- (d) **A separate load step** returning a handle carrying the roots.

Decision. Every generated module exposes `val all : db -> t list`, returning every row of its table.

Rationale. The simplest and most uniform of the four: every table is reachable without navigating to it. Option (c) would leave whole tables reachable only by navigation, ruling out queries that range over a table independently of its parents. Option (d) is subsumed by S21.2, which already supplies the handle.

Known limitation. `all` materializes the entire table, which is what the design note’s §3.2 argues against. Option (b) is the natural revision, though changing `t list` to `t Seq.t` would break consumers rather than extend them.

S21.6. The child's back-reference in the record type

A child table from S13 carries `parent_id` and `idx` columns. This asks whether they appear in the child's record.

Options considered.

- (a) **Hide both**, as encoding artifacts.
- (b) **Expose both as fields.** ← **selected**
- (c) **Expose via accessors** — `val parent : db -> t -> Root.t` — making navigation symmetric.
- (d) **Expose idx only.**

Decision. Both appear in the record, because both are columns.

```
module Item : sig
  type id
  type t = { id : id; root_id : Root.id; idx : int; sku : string; qty : int }
  val get : db -> id -> t
  val all : db -> t list
end
```

Rationale. The governing principle again. It also makes S21.5's choice pay off: with `all` returning every item from every parent flattened together, a consumer could otherwise not tell which parent a row belonged to or where it sat in the array.

Note that `idx` is not purely an artifact in the way a generated identifier is. Array order is genuine document data, and `idx` is how it is represented, so hiding it would discard something the document stated.

Known limitation. Generated identifiers, back-references and ordering indices are now all public. The interface therefore exposes the encoding, and a consumer can write code that depends on it — for instance sorting by `idx` directly rather than relying on the order `items` returns.

S22. Name mangling and collisions

Tables are named after the field whose value they hold, so two parents using the same member name want the same table:

```
{ "user": { "profile": { "name": "ada" } },
  "company": { "profile": { "industry": "rail" } } }
```

Two tables named `profile`, with different columns.

Options considered.

- (a) **Qualify by path** — `user_profile`, `company_profile`. Always unambiguous; names grow with nesting depth. ← **selected**
- (b) **Suffix on collision** — `profile`, `profile_2`. Short, but the numbering depends on traversal order, so adding an unrelated field can renumber existing modules.
- (c) **Merge structurally identical tables**, qualifying only when they differ.
- (d) **Reject on collision**, requiring an annotation to disambiguate.

Decision. Qualify by path. A table’s name is its path from the document root, with each segment mangled by S11’s scheme and the segments joined by `.`. The module name is that name with its first character capitalized.

```
. -> table root module Root
.user.profile -> table user_profile module User_profile
.company.profile -> table company_profile module Company_profile
.order.items[] -> table order_items module Order_items
```

Array brackets contribute no segment: the element table is named for the field holding the array. No ambiguity arises from this, because a path cannot be both an array and an object (S18 would reject it), and arrays of arrays are rejected outright (S17). Where S19 folds two paths together, the table takes the name of the ancestor path, which is the one that survives the unification.

Collisions cannot occur. Under (a) the scheme is injective by construction, so option (d) never fires. Three earlier decisions compose to give this:

- Paths are distinct by definition, so distinct tables have distinct paths.
- S11’s mangling is injective on each segment.
- S11 escapes `.` itself, as `_5f`. A literal underscore in a member name therefore cannot be confused with the separator joining two segments: the single field `"a.b"` mangles to `a_5fb`, while the path `.a.b` joins to `a.b`.

Capitalization is injective too. Module names must begin with an uppercase letter, and naive capitalization is not injective — the distinct members `b` and `B` would both give module `B`. S11 already resolves this before it arises. Its second rule prefixes `f_` to anything not beginning with a lowercase letter, so `B` mangles to `f_B` while `b` mangles to `b`. Every mangled name therefore begins with a lowercase ASCII letter, and capitalizing that one character is a bijection on the set of possible names:

```
member "b" -> b -> module B
member "B" -> f_B -> module F_B
```

Rationale. Path qualification is purely structural. A table’s name depends only on where it sits in the schema, not on what else the corpus happens to contain, which is the property protected in S9, S19 and S20. Option (b) fails it — the numbering depends on traversal order, so adding a field elsewhere can rename an existing module and break code written against it — and option (c) fails it more seriously, since whether two tables merge would depend on how much of the corpus had been seen.

Known limitation. Names grow with nesting depth, and the Delivery note establishes that these modules are read by people in an editor rather than consumed as a build artifact. A field four levels down becomes `User_profile_address_geo`, which is accurate and unpleasant.

Option (c) produces markedly nicer schemas and remains available, but adopting it would mean accepting that a table’s name depends on the rest of the corpus. A middle path, not adopted here, would be to allow the configuration file to supply an explicit name for a path, in the same manner as the map and recursion annotations of S9 and S19 — keeping generation structural while letting a person shorten the names that matter.

9 Accepted fragment

This section states in one place what the twenty-two decisions above amount to: exactly which inputs Phase 1 accepts, and what it reports for each input it does not. It is the specification

the implementation is written against, and the acceptance condition is also the ingest-time validator.

Acceptance is a property of a *pair*: an input and a configuration. Nothing here is a property of the input alone.

A1. Inputs and configuration

The input is a single JSON value, and must be either

- an **object**, read as a corpus of one document; or
- an **array**, read as a corpus of its elements, every one of which must be an object.

Any other top-level value is rejected, as is an array containing a non-object element. A document's root is therefore always an object (S0). An empty array is accepted and yields an empty schema.

The configuration supplies two path-directed markings, both optional:

```
maps = [".users", ".index.by_id"]
recursive = [ { path = ".children[]", folds_into = "." } ]
```

maps names object-valued paths whose members are data rather than schema (S9). **recursive** identifies a path with an ancestor path it folds into (S19).

A2. The type lattice

Inference assigns each path a type drawn from

$$\tau ::= \perp \mid \text{int} \mid \text{float} \mid \text{string} \mid \text{bool} \mid \text{obj} \mid \text{arr}(\tau)$$

ordered by $\perp \sqsubseteq \tau$ for every τ ; $\text{int} \sqsubseteq \text{float}$; $\text{arr}(\tau) \sqsubseteq \text{arr}(\tau')$ whenever $\tau \sqsubseteq \tau'$; and two object types comparable when their members are, absent members becoming partial. All other pairs are incomparable.

Nullability is not a point in this lattice. It is a separate flag on the path, raised by an explicit **null** or by the member being absent (S4), and by a null in element position (S16b).

A3. Conditions for acceptance

An input is accepted, under a configuration, exactly when all of the following hold.

- C1. Shape.** The input is an object, or an array all of whose elements are objects. (S0)
- C2. Unique members.** No object, at any depth, carries the same member name twice. (S10)
- C3. No directly nested arrays.** No array has an element that is itself an array. An array inside an object inside an array is permitted. (S17)
- C4. Type compatibility.** The types observed at every path have a least upper bound in the lattice of A2. (S16, S18)
- C5. Map markings are well placed.** Every path named in **maps** exists in the input and is object-valued. (S9)
- C6. Recursion markings are valid.** For every entry in **recursive**, both paths exist, the second is an ancestor of the first, and after inference their shapes are compatible. (S19)

C4 is the only condition that requires inference to have run; the rest are checkable on the input directly.

A4. What is accepted without complaint

Several inputs that might be expected to fail do not. Each is resolved by widening or by a collapse rather than by rejection, and the list is worth stating explicitly because it marks the boundary of C4.

- `{"a":1}` and `{"a":2.5}` — integers and floats join at `float`. (S1a)
- An integer literal too large for OCaml’s native `int` — the column widens to `float`, with the precision loss recorded in S1b rather than raised as an error. (S1b)
- A member present in some documents and absent or `null` in others — the path becomes partial. (S4)
- A member that is `null` in every document — `unit option`. (S6)
- An object that is empty in every document — a table of bare identifiers. (S8)
- An array that is empty in every document — a table with no value column. (S15)
- Objects at one path with different members — one table whose columns are partial. (S4)
- `[1, null, 2]` — a nullable element column, not a type conflict. (S16b)
- A member name that is not a legal OCaml identifier — mangled, never rejected. (S11)

A5. Rejections

Condition	Ref	Reported as
C1	S0	The top-level value is not an object or array, or an array element is not an object. Names the offending element.
C2	S10	A repeated member name. Names the path and the key. Per document, so one malformed document rejects the input.
C3	S17	An array element that is an array. Names the path.
C4	S16	Element types at one array path with no upper bound. Names the path and the types.
C4	S18	Value types at one member path with no upper bound. Names the path, the types, and the pair of documents that contributed them.
C5	S9	A <code>maps</code> entry naming a path that does not exist or is not object-valued.
C6	S19	A <code>recursive</code> entry whose paths do not exist, are not in an ancestor relation, or whose shapes are incompatible after inference. Names both paths and the mismatch.

A6. Status of the condition

It is decidable, and it is the validator. Every condition is decidable on a finite input. C1–C3, C5 and part of C6 are checkable in a single traversal; C4 and the remainder of C6 require the per-path type observations that inference computes anyway. The check is therefore not an additional pass but a reading of inference’s own intermediate state, which is what makes the same condition serviceable as the ingest-time validator for documents arriving after the schema is fixed.

It is deterministic and structural. Given an input and a configuration, acceptance and the resulting schema are fully determined. No condition depends on a threshold, a sample statistic, or a heuristic; the two places where such a rule would have been natural — map detection (S9) and recursion detection (S19) — were made configuration instead, precisely so this holds.

It is not monotone. Adding documents to an accepted corpus can make it unacceptable. Everywhere else in this document, further observations only loosen an inferred type — a column widens from `int` to `float`, or from total to partial, and a schema inferred from less data stays valid. C4 is the exception: a single new document can introduce a type conflict at a path that was uniform before, and C2 can be violated by one malformed document among many. This is the practical cost of S16 and S18, and it is recorded here rather than left to be discovered.

A7. Output

For an accepted input, Phase 1 produces

- a relational schema, with foreign-key constraints materialized and nullability recorded per column;
- the shredded data;
- the generated OCaml modules, one per table, following the two principles of S21 — the record is the row, and the store is explicit;
- the observation report of S20, giving for every path the counts of present-with-value, explicitly-null and absent, which is where the distinction S4 collapses out of the type remains visible.